

M1 DKAI

Foundations of Agent-based systems
Report

Shared Multi-City Taxi Simulation

Students :
Evan Bernier
Umair Ahmed

Professor :
Nicolas Sabouret

October 29, 2026

Contents

1 Context and objectives **2**

2 Design choices **2**

3 The environment **2**
3.1 Grid, cities and hubs . . . 2
3.2 Distance and time . . . 3
3.3 Walking . . . 3
3.4 Journey planning and multi-hop . . . 3
3.5 Passenger demand . . . 3

4 Agents and interaction **4**
4.1 Agent cycle and communication protocol . . . 4
4.2 Passenger agent . . . 4
4.3 Taxi agent: route planning . . . 4
4.4 Taxi agent: driving and boarding . . . 5
4.5 Traffic jams and emissions . . . 5
4.6 Acceptability . . . 5

5 Simulation settings and execution **5**
5.1 Parameters . . . 5
5.2 Execution of a run . . . 6
5.3 Experiments and usage . . . 6

6 Results **6**
6.1 Comparison of the configurations . . . 6
6.2 Dynamics over time . . . 7
6.3 Sensitivity to boarding time . . . 8
6.4 Sensitivity to demand . . . 8

7 Limitations and conclusion **9**

1 Context and objectives

To reduce pollution, a city council wants to test a carshare solution for taxis. Our task is to estimate the impact of such a service on **traffic**, **passenger travel time** and **acceptability**. We built a multi-agent based simulation in Python with Mesa 3, in which passengers and taxis are autonomous agents that negotiate rides through direct messages.

Following the incremental method of the brief, every feature is a switch of the same model, so each step can be compared with the previous one on exactly the same passengers:

1. **Individual**: one passenger per taxi, direct rides (the baseline).
2. **2-shared, no re-route**: two seats, but a taxi always finishes its current stop first.
3. **2-shared + re-route**: a taxi may divert on its way to pick someone up.
4. **4-shared + re-route**: four seats, direct rides (isolates the effect of capacity).
5. **4-shared + multi-hop**: trips between cities go through city hubs.
6. **4-shared + multi-hop + acceptability**: passengers may refuse offers that are too slow, too detoured or with too many transfers.

Research question: *can shared taxis reduce taxi emissions while keeping travel time and passenger satisfaction at an acceptable level, and under which conditions?* The observable variables are total and per-passenger emissions, travel time (and its parts: waiting, walking, boarding, riding), the share of distance driven with several passengers, and the share of satisfied passengers.

2 Design choices

Six principles guided the design, following the advice "make it simple":

- **A cell is a district.** Cities are small square blocks of cells on a Mesa `MultiGrid`, several agents can share a cell.
- **Distance gives time.** Taxis do not follow streets: they jump in a straight line (Euclidean distance) and the distance is converted into time. No routing algorithm is needed for cars.
- **No walking between cities.** Cities are separated by empty cells, so inter-city trips always need a taxi, while short intra-city trips can be walked.
- **Everything that costs time or pollution is a parameter** (boarding, speed, traffic, emissions), so its influence can be measured.
- **Respect the agent principles.** Agents only interact through messages and every `step()` follows the cycle *mailbox* → *perceive* → *deliberate* → *act*.
- **Fair comparison.** All passengers are generated at set-up from a dedicated random generator, so a given seed gives the same demand to every configuration, each result is averaged over 8 seeds.

Software architecture. The code has four files. `messages.py` defines the `Message` class and the performatives. `agents.py` defines `CommAgent` (a Mesa `Agent` with a mailbox) and its two subclasses `Passenger` and `Taxi`. `model.py` defines `TaxiModel` (a Mesa `Model`): it builds the grid, creates the agents, runs the step loop and records the indicators with a Mesa `DataCollector`. `run.py` is the single entry point: it reads command-line options, runs the experiments and draws the figures (matplotlib).

3 The environment

3.1 Grid, cities and hubs

Each city is a square of 4×4 districts. Cities are laid out on a square-ish array and separated by a gap of 4 empty cells (the gap must be at least 2 so that no walk can cross it). In our study there are 4 cities on a 2×2 layout, giving a 12×12 grid (Figure 1), the grid is not a torus. Each city has a **hub**,

the district closest to its centre, used as the transfer point for multi-hop trips. The hubs are at $(1, 1)$, $(9, 1)$, $(1, 9)$ and $(9, 9)$.

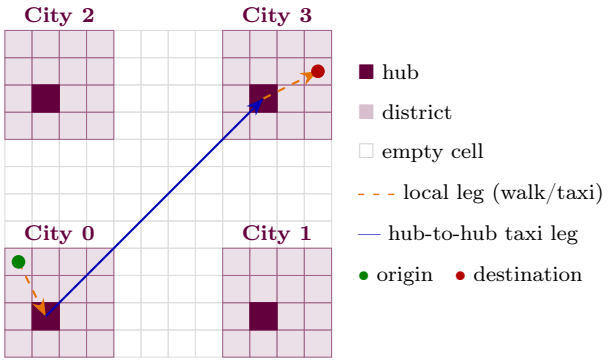


Figure 1: The 12×12 grid with 4 cities of 4×4 districts and their hubs, and a multi-hop journey from City 0 to City 3.

3.2 Distance and time

The distance between two cells is Euclidean, $d(A, B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$, and a taxi needs $d \times$ `time_per_distance` $(= 1)$ time units to cover it. The simulation advances in steps of `dt`$= 1$ time unit, but events inside a step have exact times: each taxi has a time budget of `dt` per step and spends it driving, boarding or dropping off in sequence, so a taxi that reaches a stop mid-step carries on with the remaining time. Taxis therefore have a continuous position `fpos`, their cell on the Mesa grid is that position rounded. This keeps the identity *travel time = waiting + walking + boarding + riding* exact for every passenger.

3.3 Walking

Passengers may walk between neighbouring districts of the same city. An orthogonal step costs twice its Euclidean length (2) and a diagonal step half of it (≈ 0.71). The cheapest path is found with Dijkstra's algorithm (results are cached), and walking between cities has an infinite cost. A local leg is walked if its walking time is at most `walk_max_time`$= 2.5$, otherwise a taxi is used. For example, $(0, 0) \to (2, 2)$ costs 1.41 and is walked, while $(0, 0) \to (3, 0)$ costs 3.41 and needs a taxi. Walking never pollutes.

3.4 Journey planning and multi-hop

When a passenger is created, `plan_journey` turns the trip into a list of *legs*, each being a walk or a taxi ride:

- **same city**: one leg, walked if short enough, otherwise by taxi.
- **different cities, multi-hop off**: one direct taxi leg from door to door.
- **different cities, multi-hop on**: origin → origin hub (walk or taxi), hub → hub (taxi), destination hub → destination (walk or taxi). A local leg disappears when the passenger is already at the hub.

The idea of multi-hop is to gather passengers on the long hub-to-hub leg so that it can be shared. Its cost is extra walking, extra boardings and possible transfers: with multi-hop, in a typical run about 93 of the 200 passengers only walk, 82 take one taxi, 21 take two and 4 take three.

3.5 Passenger demand

All 200 passengers are created **once, at set-up**, never during `step()`. Each one gets a random origin district, a destination in another city with probability `inter_city_prob`$= 0.3$ (otherwise another

district of the same city) and a departure time drawn uniformly in $[0, 250]$. Before its departure time a passenger is "scheduled" and invisible, it then appears on the grid and starts its journey. These draws use a dedicated generator seeded with the run's seed, so the 6 configurations face the same 200 trips; the taxi's starting districts also depend only on the seed.

4 Agents and interaction

4.1 Agent cycle and communication protocol

Every agent inherits a mailbox from `CommAgent`, `send()` appends a `Message` (performative, sender, receiver, content, time) to the receiver's mailbox, and messages are only read at the receiver's next activation. Each `step()` reads the mailbox first, then calls `perceive` (update beliefs), `deliberate` (choose an action, without changing the world) and `act` (send messages, move, board). Taxis are booked with a light Contract Net (Figure 2). Because passengers are activated before taxis in a step, a booking costs one to two steps of communication delay, which is part of the measured waiting time.

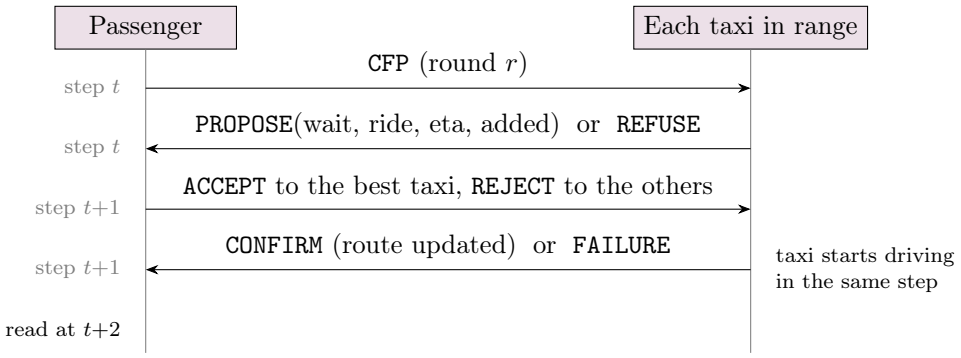


Figure 2: Booking protocol. Every message carries the round number r, so late answers to an older call are ignored.

4.2 Passenger agent

A passenger goes through the states *scheduled* → *planned* → *walking / waiting / riding* → *completed*, one leg after the other. For a walking leg it simply waits until the walk is over. For a taxi leg it sends a `CFP` to every taxi within `communication_range` (unlimited by default). When all taxis have answered, or after one step, it chooses among the proposals the one with the **earliest arrival time**, ties being broken by the smallest detour for the taxi. If every taxi refused, it calls again after `retry_delay`= 2, if the chosen taxi answers `FAILURE` (its route changed meanwhile), it calls again immediately. The taxi notifies the passenger at the exact moments of pickup, end of boarding and drop-off, which fill the four time counters. After its last leg the passenger reports its statistics to the model and removes itself from the simulation.

4.3 Taxi agent: route planning

A taxi has a `capacity` (1 to 4), a list of passengers on board and a **route**: an ordered list of stops (*pickup* or *drop-off* of a given passenger). For each request, `_best_insertion` tries every pair of positions ($i \leq j$) for the new pickup and drop-off, keeps the plans where the load never exceeds the capacity, and chooses the one that adds the least driving distance. This cheapest-insertion rule is the core of carsharing:

- with `route_change=True` the new stops may be placed anywhere, even before the stop the taxi is currently heading to, so the taxi can divert to pick someone up on its way.
- with `route_change=False` the next stop is committed and new stops go after it.

- with capacity 1 the load constraint forces each new pickup after the current drop-off, so the taxi serves its passengers one after the other.

A taxi that already has work refuses if the extra distance exceeds `max_detour`= 3. For a feasible request it answers `PROPOSE` with the waiting time, ride time, arrival time and added distance estimated along the new route. In `deliberate`, `ACCEPT` messages are handled before new `CFP`s, because they change the route the next proposals are computed on.

4.4 Taxi agent: driving and boarding

In `act` the taxi spends its time budget along its route: it drives towards the first stop (possibly only part of the way), drops off passengers instantly, and spends `boarding_time` (= 0.5) at each pickup, possibly over several steps. Distance driven, distance driven with at least two passengers on board ("shared distance"), boarding time and operating time are accumulated. A taxi with no stop left stays where it is.

4.5 Traffic jams and emissions

Traffic is local: a taxi's speed is divided by $1 + \alpha n$, where n is the number of other taxis with work in the same cell and α =`congestion_alpha`= 0.3. The time lost compared with free flow is the congestion delay. Only taxis pollute, through three components:

$$E = \underbrace{d \cdot e_d}_{\text{distance}} + \underbrace{t_{board} \cdot e_{idle}}_{\text{boarding}} + \underbrace{t_{jam} \cdot e_{idle}}_{\text{congestion}}, \qquad e_d = 1,\ e_{idle} = 0.5.$$

4.6 Acceptability

A passenger is counted as **satisfied** if no single wait exceeded `max_wait`= 8, its riding time was at most `max_ride_stretch`= 1.6 times the direct ride, and it needed at most `max_transfers`= 1 change of taxi. This indicator is measured in every configuration. When the `acceptability` switch is on, passengers also *act* on it: they ignore proposals that break the waiting or riding limit (and call again later), and replace a plan with too many transfers by a direct taxi ride.

5 Simulation settings and execution

5.1 Parameters

Table 1 lists the settings of all the experiments, unless a sweep changes one of them.

Parameter	Value	Meaning
`n_cities`, `city_side`, `city_gap`	4, 4, 4	4 cities of 4×4 districts, 4 empty cells apart
`n_taxis`	8	fleet size, random starting districts
`n_passengers`, `demand_horizon`	200, 250	demand, departures uniform in $[0, 250]$
`inter_city_prob`	0.3	share of trips to another city
`dt`, `time_per_distance`	1, 1	step length, taxi time per distance unit
`walk_max_time`	2.5	longest leg that is walked
`boarding_time`	0.5	time per boarding (swept from 0 to 3)
`emission_per_distance`, `idle_emission_rate`	1, 0.5	emission rates
`congestion_alpha`	0.3	strength of traffic jams
`max_detour`, `retry_delay`	3, 2	taxi detour limit, delay before a new call
`max_wait`, `max_ride_stretch`, `max_transfers`	8, 1.6, 1	acceptability thresholds
seeds, `max_steps`	1–8, 2000	repetitions, safety limit of a run

Table 1: Simulation parameters.

5.2 Execution of a run

`TaxiModel.step()` activates all passengers in random order (`shuffle_do`), then all taxis in random order, advances the clock by `dt` and records the indicators with the `DataCollector`. A run stops as soon as every passenger has arrived (here after about 270 steps), with `max_steps` as a safety limit. Indicators are averages over *completed* passengers or cumulative taxi totals.

5.3 Experiments and usage

`run.py` is the only file to run. Without arguments it performs the whole study: the 6 configurations × 8 seeds, then two **sweeps** that vary one scenario parameter while keeping the others fixed, for three key configurations (Individual, 2-shared + re-route, 4-shared + multi-hop): boarding time in $\{0, 0.5, 1, 2, 3\}$ and number of passengers in $\{50, 100, 200, 300\}$. This makes 264 simulations, about half a minute in total. Figures and CSV tables are written to `results/`. With options, a single configuration is run, for example:

```
python run.py -capacity 2 -no-reroute -no-multi-hop -passengers 300 -taxis 5
```

The policy switches are `-shared` / `-no-shared`, `-capacity`, `-reroute` / `-no-reroute`, `-multi-hop` / `-no-multi-hop` and `-acceptability`, the scenario options include `-passengers`, `-taxis`, `-cities`, `-boarding-time` and `-seeds`.

6 Results

Bars are means over 8 seeds and error bars are standard deviations, 200 passengers, 8 taxis, 4 cities and a boarding time of 0.5 unless stated otherwise. Every run served 100% of passengers.

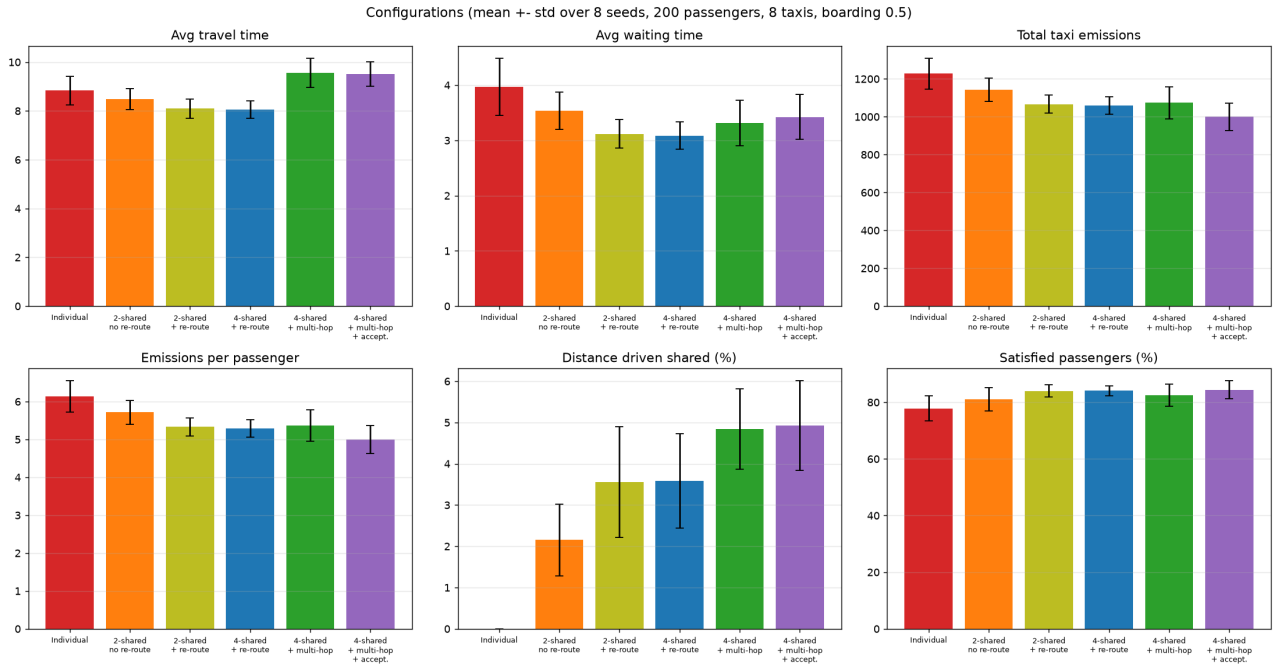


Figure 3: Comparison of the six configurations (`fig_configs.png`).

6.1 Comparison of the configurations

Emissions. Sharing lowers total emissions from 1227 (Individual) to 1143 with two seats and 1066 with re-routing (−13%), four seats add almost nothing (1058). Per passenger, emissions fall from 6.14

to 5.33. Surprisingly, the share of distance driven with two passengers or more stays small (2–5%), and mean occupancy only rises from 0.32 to 0.34 passengers per taxi: with 8 taxis for 200 trips spread over 4 cities and 250 time units, two compatible trips rarely coincide. The gain mostly comes from **less empty driving**. With one seat, a busy taxi can only offer a late pickup, so the passenger often books a distant free taxi that drives empty across the city, a shared taxi with re-routing can insert a nearby pickup into its current route. We measured that about 42% of the distance is driven empty for Individual, against 35% with re-routing. Distance accounts for about 97% of emissions (Figure 4).

Travel and waiting time. Travel time falls from 8.83 to 8.09 with re-routing, mainly because waiting drops from 3.97 to 3.12. Multi-hop has the opposite effect: travel time rises to 9.56 because walking to and from the hubs almost doubles (1.14 to 2.03) and each transfer adds a boarding and a new wait.

Multi-hop. Hubs raise the shared distance to 4.8% but do not reduce the total distance (1033 against 1028), and they add boarding emissions (34 against 27) and congestion, since taxis meet at the hubs (congestion delay 12.3 against 7.1). With 4 cities there are 12 hub-to-hub directions, so the long leg rarely finds a partner at the right time.

Acceptability. The filter gives the lowest emissions (1000 in total, 5.00 per passenger, −19% against Individual) and the highest satisfaction (84.4% against 77.8%). On average 30 offers per run are refused and 4 passengers replace a plan with two transfers by a direct ride. Refusing long waits seems to discard offers from distant taxis, so empty driving drops to about 28%. Travel time stays high (9.52) because the walk to the hubs remains, and the negotiation needs more messages (3639 against 2548 with re-routing).

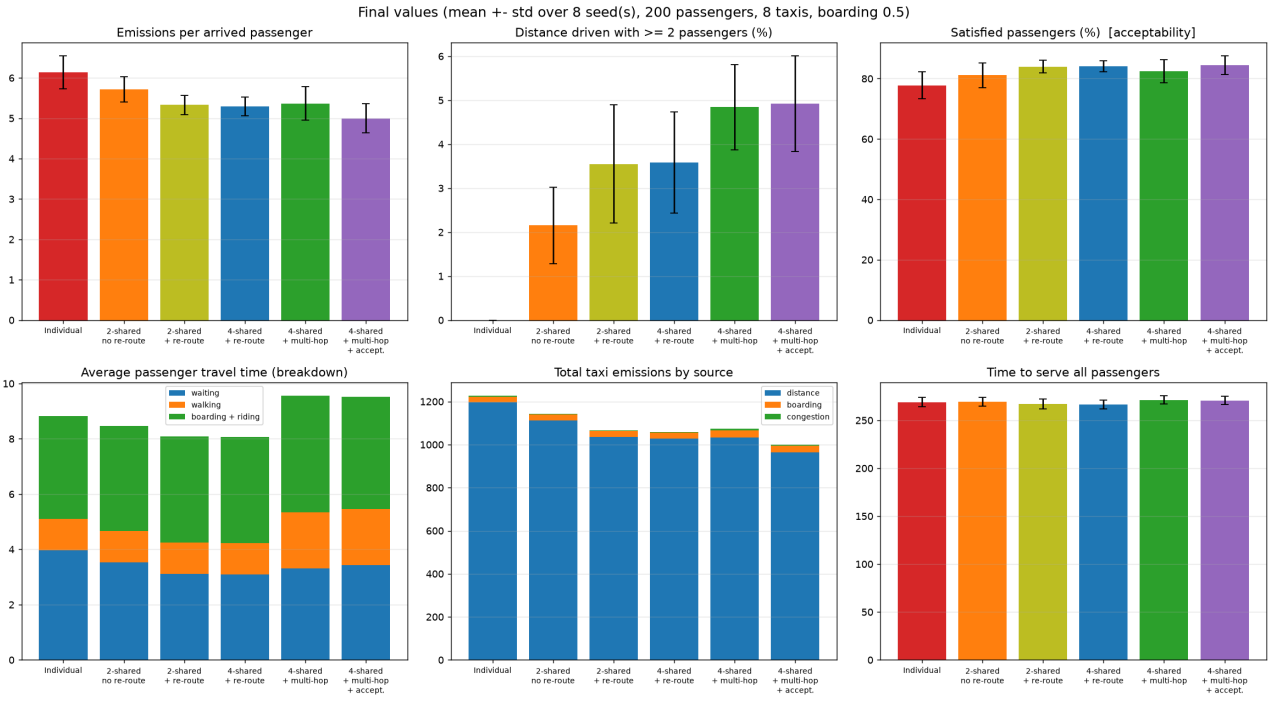


Figure 4: Travel-time and emission breakdowns and time to serve all passengers (`fig_breakdown.png`).

6.2 Dynamics over time

Figure 5 shows that the system is stable at this load: between 5 and 9 passengers are waiting or travelling at any time, arrivals follow the demand closely and every configuration finishes around step

270, just after the last departure. Cumulative emissions diverge steadily, so the savings are not due to a particular moment of the run, and the cumulative congestion delay is clearly highest with multi-hop.

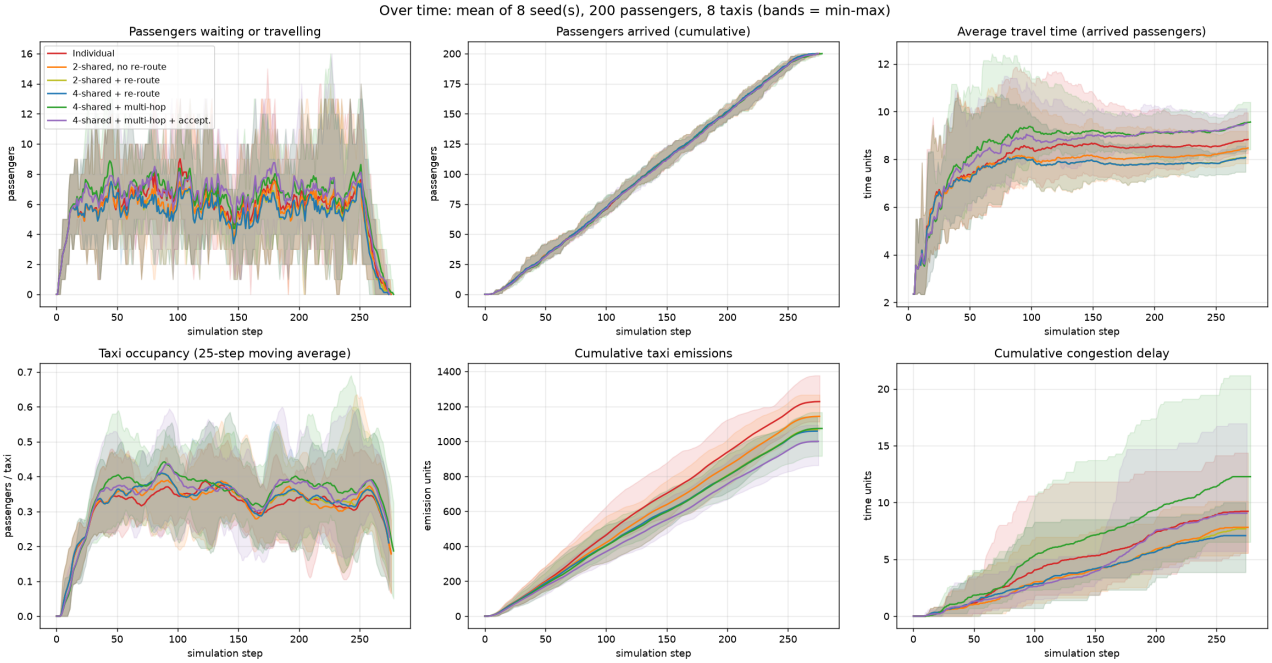


Figure 5: Evolution over time, mean of 8 seeds, bands from minimum to maximum (`fig_timeseries.png`).

6.3 Sensitivity to boarding time

From 0 to 3 time units of boarding (Figure 6), travel time rises by 50% for Individual (8.2 to 12.3), 40% for 2-shared (7.7 to 10.8) and 46% for multi-hop (9.1 to 13.3). Emissions per passenger rise in the same way (5.8 to 7.3 for Individual), since boarding keeps the engine idling. The ranking never changes: 2-shared with re-routing is always the fastest, and the emission advantage of multi-hop over 2-shared vanishes at 3 time units because it needs more boardings.

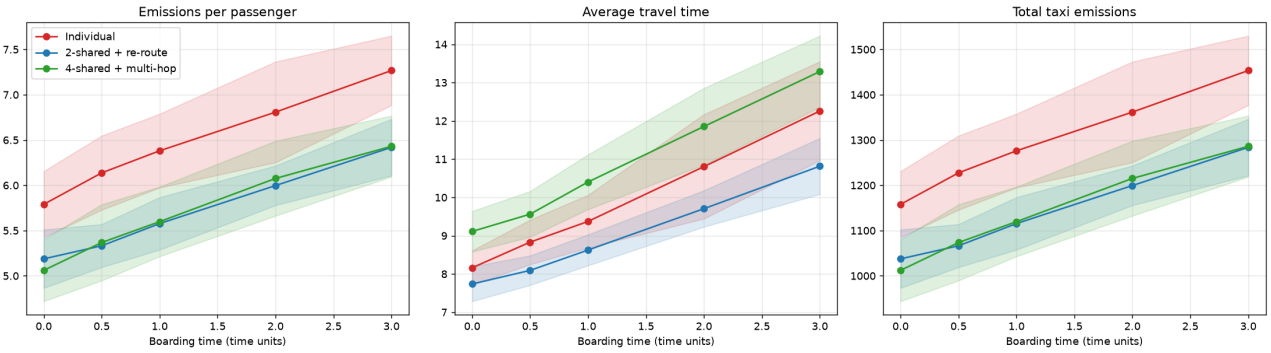


Figure 6: Boarding-time sweep, mean ± standard deviation over 8 seeds (`fig_boarding.png`).

6.4 Sensitivity to demand

Demand is the decisive factor (Figure 7). With 50 or 100 passengers taxis are mostly free: Individual and 2-shared are equivalent (travel time 7.0, emissions per passenger 4.8) and multi-hop is only slower. With 300 passengers the individual fleet saturates: waiting jumps to 17.4 and travel time to 22.4, against 10.6 for 2-shared and 11.7 for multi-hop. Sharing then cuts total emissions by 21% (2084 to 1640) and multi-hop by 25% (1555), at this load multi-hop gives the lowest emissions per passenger (5.18), because enough passengers travel between the same hubs.

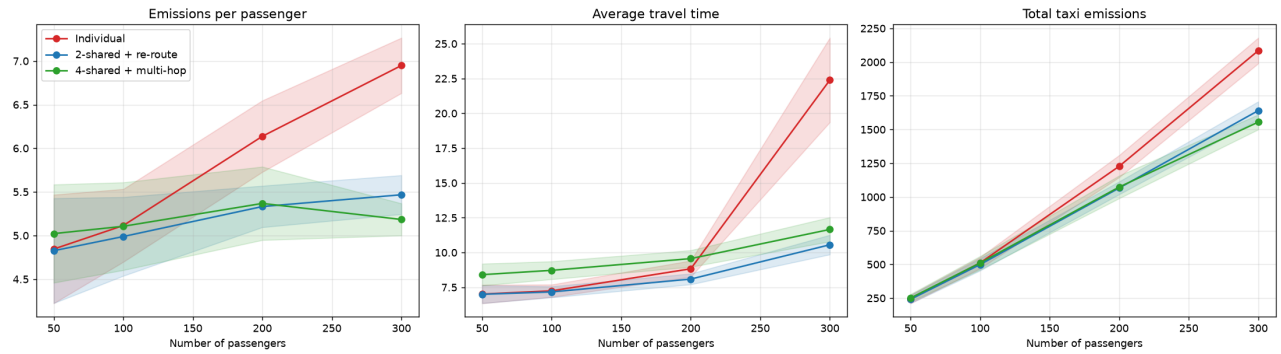


Figure 7: Demand sweep, mean ± standard deviation over 8 seeds (`fig_passengers.png`).

7 Limitations and conclusion

Limitations. Taxis move in straight lines and idle taxis do not reposition; demand is uniform in space and time, traffic jams only depend on taxis in the same cell, the acceptability thresholds are assumptions, not survey data, and results rest on 8 seeds of one map. These choices keep the model simple and can each be refined through the existing parameters.

Conclusion. Shared taxis with re-routing reduce emissions by about 13% and slightly reduce travel time, with more satisfied passengers, on the same demand as individual taxis, adding an acceptability filter brings the saving to 19%. At moderate demand the gain comes from less empty driving rather than from passengers riding together. Sharing is robust to slow boarding and becomes decisive at high demand, where individual taxis saturate and sharing cuts emissions by 21–25% while halving travel time, at low demand it brings nothing. Re-routing is the most useful feature, a larger capacity adds little with 8 taxis, and multi-hop trades longer journeys for an emission saving that only appears under high demand. The city council should therefore deploy carsharing with re-routing, target it at busy periods, and use hubs only when inter-city demand is high.